

Correspondence between NUTS implemented by hand and the Gellmann's article

Tommaso Pirazzo

In the following there is the 6th algorithm of the article "*The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo*" with the indicated correspondences with the code implemented in the file `JAX_NUTS_functions.ipynb`.

1 Leapfrog integrator

Algorithm 1: Leapfrog Integrator

Input: Position θ , momentum r , step size ϵ

Output: Updated (θ, r)

$\tilde{r} \leftarrow r + \frac{\epsilon}{2} \nabla_{\theta} L(\theta)$

// $L(\theta) = \log p(\theta)$

$\tilde{\theta} \leftarrow \theta + \epsilon \tilde{r}$

$\tilde{r} \leftarrow \tilde{r} + \frac{\epsilon}{2} \nabla_{\theta} L(\tilde{\theta})$ **return** (θ, r)

```
1 def Leapfrog(theta, r, epsilon):
2     # half-step momentum
3     p = r + 0.5 * epsilon * grad_L(theta)
4
5     # full-step position
6     q = theta + epsilon * p
7
8     # half-step momentum
9     p = p + 0.5 * epsilon * grad_L(q)
10
11     return q, p
```

2 FindReasonableEpsilon

Algorithm 2: Heuristic for choosing an initial value of ϵ

Input: Current position θ
Output: Initial step size ϵ
Sample momentum $r \sim \mathcal{N}(0, I)$
 $\epsilon \leftarrow 1$
 $E_0 \leftarrow \log p(\theta, r)$
 $(\theta', r') \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$
 $E_1 \leftarrow \log p(\theta', r')$
 $\log_accept \leftarrow E_1 - E_0$
if $\log_accept > \log(0.5)$ **then**
 $a \leftarrow +1$
else
 $a \leftarrow -1$
while $a \cdot \log_accept > -a \cdot \log(2)$ **do**
 $\epsilon \leftarrow \epsilon \cdot 2^a$
 $(\theta', r') \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$
 $E_1 \leftarrow \log p(\theta', r')$
 $\log_accept \leftarrow E_1 - E_0$
return ϵ

```
1 def FindReasonableEpsilon(theta, key) :
2     assert theta.ndim == 1
3
4     # sample momentum r ~ N(0, I)
5     key, subkey = jrnd.split(key)
6     r = jrnd.normal(subkey, shape=theta.shape)
7
8     # initial epsilon
9     epsilon = 1.0
10
11     # compute initial log joint density
12     E0 = log_p(theta, r)
13
14     # one leapfrog step
15     theta_prime, r_prime = Leapfrog(theta, r, epsilon)
16     E1 = log_p(theta_prime, r_prime)
17
18     # log acceptance probability
19     log_accept = E1 - E0
```

```

20
21 # direction: +1 (increase) or -1 (decrease)
22 # equivalent to the "if" logic of the pseudo-code
23 a = jnp.where(log_accept > jnp.log(0.5), 1.0, -1.0)
24
25 # -----
26 # cond_fun: while loop condition
27 # -----
28 def cond_fun(state):
29     eps, log_acc = state
30     return (a * log_acc) > (-a * jnp.log(2.0))
31
32 # -----
33 # body_fun: loop body
34 # -----
35 def body_fun(state):
36     eps, _ = state
37     eps = eps * (2.0 ** a)
38     theta_p, r_p = Leapfrog(theta, r, eps)
39     E1 = log_p(theta_p, r_p)
40     log_acc = E1 - E0
41     return (eps, log_acc)
42
43 # -----
44 # while loop in JAX
45 # -----
46 epsilon, _ = jax.lax.while_loop(cond_fun, body_fun,
47     (epsilon, log_accept))
48
49 return epsilon

```

3 BuildTree

This pseudo-code is referred to the **BuildTree** function implemented in the **Algorithm 6** (No-U-Turn Sampler with Dual Averaging) of the article. This function is composed by two cases:

- $j = 0$: Base case $\rightarrow$ take one leapfrog step in the direction v .
- $j \neq 0$: Recursion $\rightarrow$ implicitly build the left and right subtrees.

The "base case" is implemented before in the function `build_tree_base` and then called in the complete function `BuildTree`.

Algorithm 3: BuildTree($\theta, r, u, v, j, \epsilon, \theta_0, r_0$)

Input: Position θ , momentum r , slice variable u , direction $v \in \{-1, +1\}$, depth j , step size ϵ , reference state (θ_0, r_0)

Output: (θ^-, r^-) , (θ^+, r^+) , candidate θ' , number of valid points n' , stop flag s' , acceptance sum α' , acceptance count n'_α

// Base case

if $j = 0$ **then**

// build_tree_base

$(\theta', r') \leftarrow \text{Leapfrog}(\theta, r, v \cdot \epsilon)$

$E' \leftarrow \log p(\theta', r')$

$n' \leftarrow \mathbb{I}[\log u \leq E']$

$s' \leftarrow \mathbb{I}[\log u < \Delta_{\max} + E']$

$\Delta \leftarrow E' - \log p(\theta_0, r_0)$

$\alpha' \leftarrow \min\{1, \exp(\Delta)\}$

$n'_\alpha \leftarrow 1$

return (θ', r') , (θ', r') , θ' , n' , s' , α' , n'_α

// Recursive case

$(\theta^-, r^-), (\theta^+, r^+), \theta', n', s', \alpha', n'_\alpha \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon, \theta_0, r_0)$

if $s' = 0$ **then**

return (θ^-, r^-) , (θ^+, r^+) , $\theta', n', s', \alpha', n'_\alpha$

if $v = -1$ **then**

$(\theta^-, r^-), (-, -), \theta'', n'', s'', \alpha'', n''_\alpha \leftarrow$

$\text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon, \theta_0, r_0)$

else

$(-, -), (\theta^+, r^+), \theta'', n'', s'', \alpha'', n''_\alpha \leftarrow$

$\text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon, \theta_0, r_0)$

$p \leftarrow \frac{n''}{n' + n''}$ **if** Bernoulli(p) **then**

$\theta' \leftarrow \theta''$

$n' \leftarrow n' + n''$

$s' \leftarrow s' \cdot s'' \cdot \text{Turning}(\theta^-, r^-, \theta^+, r^+)$

$\alpha' \leftarrow \alpha' + \alpha''$

$n'_\alpha \leftarrow n'_\alpha + n''_\alpha$

return (θ^-, r^-) , (θ^+, r^+) , $\theta', n', s', \alpha', n'_\alpha$

```
1 def build_tree_base(root: Root) -> Tree:
2     theta, r = root.theta, root.r
3     u, v = root.u, root.v
4     eps = root.epsilon
5     theta0, r0 = root.theta_0, root.r_0
6
```

```

7   # leapfrog step
8   theta_p, r_p = Leapfrog(theta, r, v * eps)
9
10  logu = jnp.log(u)
11  lp = log_p(theta_p, r_p)
12
13  # valid point?
14  n_p = (logu <= lp).astype(jnp.int32)
15
16  # non-divergent?
17  s_p = (logu < (1000.0 + lp)).astype(jnp.int32)
18
19  # acceptance statistic
20  delta = (
21      L(theta_p) - 0.5 * jnp.dot(r_p, r_p)
22      - L(theta0) + 0.5 * jnp.dot(r0, r0))
23
24  alpha_p = jnp.minimum(1.0, jnp.exp(delta))
25  n_a_p = jnp.array(1, dtype=jnp.int32)
26
27  return Tree(
28      theta_minus=theta_p,
29      r_minus=r_p,
30      theta_plus=theta_p,
31      r_plus=r_p,
32      theta_prime=theta_p,
33      n_prime=n_p,
34      s_prime=s_p,
35      alpha_prime=alpha_p,
36      n_a_prime=n_a_p,
37  )

```

```

1  def BuildTree(root: Root, key: jnp.ndarray):
2
3      def build_base(_):
4          return build_tree_base(root), key
5
6      def build_iter(_):
7
8          base_tree = build_tree_base(root)
9
10         state = TreeState(

```

```

11     theta_minus=base_tree.theta_minus,
12     r_minus=base_tree.r_minus,
13     theta_plus=base_tree.theta_plus,
14     r_plus=base_tree.r_plus,
15     theta_prime=base_tree.theta_prime,
16     n_prime=base_tree.n_prime,
17     s_prime=base_tree.s_prime,
18     alpha_prime=base_tree.alpha_prime,
19     n_a_prime=base_tree.n_a_prime,
20     key=key,
21 )
22
23 # body function of the iterative case
24 def body_fun(depth, state: TreeState):
25
26     def expand_valid(state):
27
28         theta_start = jnp.where(root.v == -1,
29                                 state.theta_minus, state.theta_plus)
30         r_start = jnp.where(root.v == -1,
31                             state.r_minus, state.r_plus)
32
33         local_root = Root(
34             theta=theta_start,
35             r=r_start,
36             u=root.u,
37             v=root.v,
38             j=0,
39             epsilon=root.epsilon,
40             theta_0=root.theta_0,
41             r_0=root.r_0,
42         )
43
44         local_tree = build_tree_base(local_root)
45
46         theta_minus = jnp.where(root.v==-1, local_tree.theta_minus,
47                                 state.theta_minus)
48         r_minus = jnp.where(root.v==-1, local_tree.r_minus,
49                             state.r_minus)
50         theta_plus = jnp.where(root.v==1, local_tree.theta_plus,
51                                state.theta_plus)
52         r_plus = jnp.where(root.v == 1, local_tree.r_plus,
53                             state.r_plus)

```

```

54
55     total_n = state.n_prime + local_tree.n_prime
56     p = jnp.where(total_n > 0,
57                    local_tree.n_prime / total_n, 0.5)
58
59     key, sub = jrnd.split(state.key)
60     choose = jrnd.bernoulli(sub, p)
61
62     theta_prime = jnp.where(choose,
63                             local_tree.theta_prime,
64                             state.theta_prime)
65
66     n_prime = state.n_prime + local_tree.n_prime
67     s_prime = state.s_prime *
68               local_tree.s_prime *
69               stop_criterion(theta_minus, theta_plus, r_minus, r_plus)
70     alpha_prime = state.alpha_prime + local_tree.alpha_prime
71     n_a_prime = state.n_a_prime + local_tree.n_a_prime
72
73     return TreeState(
74         theta_minus=theta_minus,
75         r_minus=r_minus,
76         theta_plus=theta_plus,
77         r_plus=r_plus,
78         theta_prime=theta_prime,
79         n_prime=n_prime,
80         s_prime=s_prime,
81         alpha_prime=alpha_prime,
82         n_a_prime=n_a_prime,
83         key=key,
84     )
85
86     return jax.lax.cond(
87         state.s_prime == 1,
88         expand_valid,
89         lambda s: s,
90         operand=state,
91     )
92
93     final_state = jax.lax.fori_loop(1, root.j + 1, body_fun, state)
94
95     merged = Tree(
96         theta_minus=final_state.theta_minus,

```

```

97         r_minus=final_state.r_minus,
98         theta_plus=final_state.theta_plus,
99         r_plus=final_state.r_plus,
100        theta_prime=final_state.theta_prime,
101        n_prime=final_state.n_prime,
102        s_prime=final_state.s_prime,
103        alpha_prime=final_state.alpha_prime,
104        n_a_prime=final_state.n_a_prime,
105    )
106
107    return merged, final_state.key
108
109    # More external "if" condition to select the base/iterative case
110    return jax.lax.cond(
111        root.j == 0,
112        build_base,
113        build_iter,
114        operand=None
115    )

```

Algorithm 4: BuildTree (versione iterativa)

Input: State (θ, r) , slice variable u , direction v , depth j , step size ϵ ,
reference (θ_0, r_0)

Output: $(\theta^-, r^-), (\theta^+, r^+), \theta', n', s', \alpha', n'_\alpha$

if $j = 0$ **then**

$(\theta^-, r^-), (\theta^+, r^+), \theta', n', s', \alpha', n'_\alpha \leftarrow \text{BASETREE}(\theta, r, u, v, \epsilon, \theta_0, r_0)$

return $(\theta^-, r^-), (\theta^+, r^+), \theta', n', s', \alpha', n'_\alpha$

$(\theta^-, r^-), (\theta^+, r^+), \theta', n', s', \alpha', n'_\alpha \leftarrow \text{BASETREE}(\theta, r, u, v, \epsilon, \theta_0, r_0)$

for $\ell \leftarrow 1$ **to** j **do**

if $s' = 0$ **then**

break

if $v = -1$ **then**

$\theta_{\text{start}} \leftarrow \theta^-$

$r_{\text{start}} \leftarrow r^-$

else

$\theta_{\text{start}} \leftarrow \theta^+$

$r_{\text{start}} \leftarrow r^+$

$(\theta_{\text{loc}}^-, r_{\text{loc}}^-), (\theta_{\text{loc}}^+, r_{\text{loc}}^+), \theta'_{\text{loc}}, n'_{\text{loc}}, s'_{\text{loc}}, \alpha'_{\text{loc}}, n_{\alpha, \text{loc}} \leftarrow$
 $\text{BASETREE}(\theta_{\text{start}}, r_{\text{start}}, u, v, \epsilon, \theta_0, r_0)$

if $v = -1$ **then**

$\theta^- \leftarrow \theta_{\text{loc}}^-$

$r^- \leftarrow r_{\text{loc}}^-$

else

$\theta^+ \leftarrow \theta_{\text{loc}}^+$

$r^+ \leftarrow r_{\text{loc}}^+$

$n_{\text{tot}} \leftarrow n' + n'_{\text{loc}}$

$p \leftarrow n'_{\text{loc}} / n_{\text{tot}}$

if $\text{BERNOULLI}(p)$ **then**

$\theta' \leftarrow \theta'_{\text{loc}}$

$n' \leftarrow n_{\text{tot}}$

$s' \leftarrow s' \cdot s'_{\text{loc}} \cdot \text{STOPCRITERION}(\theta^-, \theta^+, r^-, r^+)$

$\alpha' \leftarrow \alpha' + \alpha'_{\text{loc}}$

$n'_\alpha \leftarrow n'_\alpha + n_{\alpha, \text{loc}}$

return $(\theta^-, r^-), (\theta^+, r^+), \theta', n', s', \alpha', n'_\alpha$

4 Final algorithm

Algorithm 5: No-U-Turn Sampler with Dual Averaging

Given θ_0 , δ , L , M , M^{adapt} :

// **FindReasonableEpsilon**(θ , **key**) $\rightarrow$ **epsilon**

Set $\epsilon_0 = \text{FindReasonableEpsilon}(\theta)$,

$\mu = \log 10\epsilon_0$, $\bar{\epsilon}_0 = 1$, $\bar{H}_0 = 0$, $\gamma = 0.05$, $t_0 = 10$, $k = 0.75$.

Input: Array A di lunghezza n , valore x

Output: Indice i tale che $A[i] = x$, oppure **None**

for $i \leftarrow 1$ **to** n **do**

if $A[i] = x$ **then**
 return i

return **None**
